```
int floor_sum(int n, int m, int a, int b) {
    int ans = 0;
    while (true) {
        if (a >= m) {
        ans += (n - 1) * n / 2 * (a / m);
        a %= m;
    }
    if (b >= m) {
        ans += n * (b / m);
        b %= m;
    }

    int y_max = a * n + b;
    if (y_max < m) break;
    n = (y_max / m);
    b = (y_max % m);
    swap(m, a);
    }
    return ans;
}
```

这是一份为 LOJ 138（要求 $X+Y \leq 10$）量身定做的**“究极类欧几里得算法”模板。**

相比于传统的、为了单一公式推导半天的代码，这份模板的最大亮点在于**“一次递归，全族降维”**：它利用了状态的封闭性（不论怎么转化，下标和 $X+Y$ 始终 $\leq K$），**在每一次递归中同时计算出所有可能的 (X,Y) 组合**，并在 $O(K^4 \log c)$ 的极低复杂度内解决战斗。

核心亮点

1. **全状态同时推导**：直接返回一个二维数组，包含所有 $X+Y \leq K$ 的答案。遇到 $a \geq c$ 时，原本分离的二项式展开被完美合并为一次多项式乘法合并。
2. **极简多项式系数预处理**：没有用晦涩难懂的拉格朗日插值或者高斯消元，而是使用**第一/二类斯特林数 $O(K^2)$** 直接展开自然数幂和多项式，绝对没有精度问题。
3. **防溢出设计**：使用 `__int128_t` 处理临界状态 $m = \lfloor \frac{an+b}{c} \rfloor$，杜绝 $10^9 \times 10^9$ 乘法爆炸。

C++ 模板代码

```
#include <iostream>
#include <cstring>
#include <vector>

using namespace std;

// === 配置区 ===
const int MOD = 1e9 + 7;
const int K_MAX = 10; // 对应题目中 X + Y <= 10 的要求

// 快速幂
```

```cpp
long long power(long long base, long long exp) {
    long long res = 1;
    base %= MOD;
    while (exp > 0) {
        if (exp % 2 == 1) res = res * base % MOD;
        base = base * base % MOD;
        exp /= 2;
    }
    return res;
}
long long inv(long long n) { return power(n, MOD - 2); }

// === 预处理区 ===
long long S2[15][15];    // 第二类斯特林数
long long P[15][15];     // P[m](n) 为 (n+1)^{\underline{m}} 的展开系数
long long polyS[15][15]; // polyS[k][i] 表示自然数 k 次幂和 S_k(n) 的 n^i 系数
long long C[15][15];     // 组合数

void init() {
    memset(S2, 0, sizeof(S2));
    S2[0][0] = 1;
    for (int i = 1; i <= 12; ++i)
        for (int j = 1; j <= i; ++j)
            S2[i][j] = (S2[i - 1][j - 1] + j * S2[i - 1][j]) % MOD;

    memset(P, 0, sizeof(P));
    P[0][0] = 1;
    for (int m = 1; m <= 12; ++m) {
        long long constant = (1 - (m - 1) + MOD) % MOD;
        for (int i = 0; i <= m; ++i) {
            if (i > 0) P[m][i] = (P[m][i] + P[m - 1][i - 1]) % MOD; // 乘 n
            P[m][i] = (P[m][i] + P[m - 1][i] * constant) % MOD;     // 乘常数
        }
    }

    memset(polyS, 0, sizeof(polyS));
    for (int k = 0; k <= K_MAX; ++k) {
        for (int j = 0; j <= k; ++j) {
            long long mult = S2[k][j] * inv(j + 1) % MOD;
            for (int i = 0; i <= j + 1; ++i) {
                polyS[k][i] = (polyS[k][i] + mult * P[j + 1][i]) % MOD;
            }
        }
    }

    memset(C, 0, sizeof(C));
    for (int i = 0; i <= 12; ++i) {
        C[i][0] = 1;
        for (int j = 1; j <= i; ++j)
            C[i][j] = (C[i - 1][j - 1] + C[i - 1][j]) % MOD;
    }
}
```

```cpp
// 快速求自然数幂和 S_k(n) = \sum_{i=0}^n i^k
long long eval_S(int k, long long n) {
    long long res = 0, cur_n = 1;
    n %= MOD;
    for (int i = 0; i <= k + 1; ++i) {
        res = (res + polyS[k][i] * cur_n) % MOD;
        cur_n = (cur_n * n) % MOD;
    }
    return res;
}

// 封装状态，由于 K_MAX=10，121 个 long long 直接按值传递，毫无压力
struct State {
    long long f[11][11];
    State() { memset(f, 0, sizeof(f)); }
};

// === 核心逻辑区 ===
State solve(long long n, long long a, long long b, long long c) {
    State cur;
    if (n < 0) return cur; // 保底防御

    // 边界 1：斜率 a 为 0，退化为常数
    if (a == 0) {
        long long qb = b / c;
        long long p_qb[12];
        p_qb[0] = 1;
        for (int i = 1; i <= K_MAX; ++i) p_qb[i] = p_qb[i - 1] * (qb % MOD) % MOD;

        for (int X = 0; X <= K_MAX; ++X) {
            long long sx = eval_S(X, n);
            for (int Y = 0; X + Y <= K_MAX; ++Y) {
                cur.f[X][Y] = sx * p_qb[Y] % MOD;
            }
        }
        return cur;
    }

    // 边界 2：截距或斜率超过了 c，将多余部分通过二项式定理提出来，问题降级
    if (a >= c || b >= c) {
        long long qa = (a / c) % MOD;
        long long qb = (b / c) % MOD;
        long long p_qa[12], p_qb[12];
        p_qa[0] = 1; p_qb[0] = 1;
        for (int i = 1; i <= K_MAX; ++i) {
            p_qa[i] = p_qa[i - 1] * qa % MOD;
            p_qb[i] = p_qb[i - 1] * qb % MOD;
        }

        State sub = solve(n, a % c, b % c, c); // 核心降级
```

```
        for (int X = 0; X <= K_MAX; ++X) {
            for (int Y = 0; X + Y <= K_MAX; ++Y) {
                long long res = 0;
                for (int j = 0; j <= Y; ++j) {
                    for (int l = 0; l <= Y - j; ++l) {
                        long long term = C[Y][j] * C[Y - j][l] % MOD;
                        term = term * p_qa[l] % MOD;
                        term = term * p_qb[Y - j - l] % MOD;
                        // 注意这里调用的子状态必定满足 (X+l)+j <= 10，状态自然封闭！
                        term = term * sub.f[X + l][j] % MOD;
                        res = (res + term) % MOD;
                    }
                }
                cur.f[X][Y] = res;
            }
        }
        return cur;
    }

    // 核心转换 (a < c 且 b < c)：反转坐标轴！
    long long m = (long long)(((unsigned __int128)a * n + b) / c);

    // 连 1 都没有跨过，全是 0
    if (m == 0) {
        for (int X = 0; X <= K_MAX; ++X) {
            cur.f[X][0] = eval_S(X, n);
            for (int Y = 1; X + Y <= K_MAX; ++Y) cur.f[X][Y] = 0;
        }
        return cur;
    }

    // 真正的坐标轴反转递归
    State sub = solve(m - 1, c, c - b - 1, a);

    for (int X = 0; X <= K_MAX; ++X) {
        for (int Y = 0; X + Y <= K_MAX; ++Y) {
            if (Y == 0) {
                cur.f[X][Y] = eval_S(X, n);
            } else {
                long long res = 0;
                long long sXn = eval_S(X, n);
                for (int k = 0; k <= Y - 1; ++k) {
                    long long inner = 0;
                    // 注意：这里 T 的枚举范围 T <= X+1
                    for (int T = 0; T <= X + 1; ++T) {
                        // 防御性越界拦截（数学上证明了必定 k+T <= 10）
                        if (k + T > K_MAX) continue;
                        long long term = polyS[X][T] * sub.f[k][T] % MOD;
                        inner = (inner + term);
                        if (inner >= MOD) inner -= MOD; // 微小常数优化
                    }
                    long long part = eval_S(k, m - 1) * sXn % MOD;
```

```
                    part = (part - inner + MOD) % MOD;
                    part = part * C[Y][k] % MOD;
                    res = (res + part) % MOD;
                }
                cur.f[X][Y] = res;
            }
        }
    }
    return cur;
}
```

使用指南

1. **启动预处理**：在主函数最开头，**切记只调用一次** `init();`，这能在 $O(K^2)$ 的时间内极其快速地为你生成好后面用到的自然数幂和系数等信息。
2. **查询接口**：不管题目问的是单一组合，还是多组组合，只需要调用 `State ans = solve(n, a, b, c);`
3. **获取答案**：返回的 `ans.f[X][Y]` 就是你要的 $\sum i^X \lfloor \frac{ai+b}{c} \rfloor^Y$ 的值。你可以通过读取对应坐标瞬间得到结果。